Candidate Coding Challenge

WebSocket Hub

Context

At Haiven, we're building tools to help game developers understand and improve the health of their communities. Much of what we are building will happen in real time. Things like chat, presence, notifications, moderation signals, and other events all need to move between players and backend services with low latency.

Rather than have every service open its own connection to each client, we want a single WebSocket Hub. Each client holds one persistent connection, and the hub routes messages between that client and multiple downstream services (chat, presence, notifications, etc…).

This exercise is meant to show how you think through a fuzzy problem, how you design systems at a high level, and how clearly you communicate your decisions.

This is *not* about perfect code or production-ready detail. Please *limit* the use of AI in answering. We are mainly interested in how you reason through a problem and communicate your solution.

The Challenge

Design a **high-level WebSocket hub** in Go that maintains a single connection per client and allows multiple downstream services to send and receive messages over that connection.

You do not need to build anything end-to-end. We care more about your reasoning than implementation polish.

What to Include

1. High-Level Design

Explain how you would approach this problem end-to-end, including:

- How clients connect, authenticate, and maintain their connection
- How messages are framed and routed to downstream services
- Your concurrency model in Go
- How downstream services integrate with the hub
- Any assumptions or constraints you're making

A diagram is welcome but optional.

2. Low-level Design

Explain how you would approach low-level architectural decisions:

- Connection lifecycle: register, unregister, read/write loop
- The message envelope and how routing decisions are made
- Backpressure handling
- Fan-out (downstream service pushing a message to one or many clients)
- Heartbeat/dead-connection detection

3. Documentation / Explanation

Write a short explanation (as if for a mixed product + engineering audience) covering:

- Key tradeoffs (delivery guarantees vs complexity, latency vs throughput, stateful hubs vs stateless routing, memory per connection)
- How to handle common edge cases (reconnects and duplicate connections, out-of-order delivery, oversized or malformed messages, a downstream service going down, slow consumers)
- How to evaluate whether the system is working well
- What to intentionally defer to a later phase

4. Working With a Team

Treat your submission like a design doc you'd share with teammates before implementation begins, not a finished product. As part of your write-up, please include:

- Questions for the backend team. Identify 1-2 decisions in your design where you would genuinely want input from other backend engineers.
- Questions for the frontend team. Identify 1-2 decisions you think would impact the frontend team or require their input.
- How you would split the work. If you and two other engineers were building this over one or two sprints, how would you divide it so people can work in parallel without stepping on each other?

For each question, state the options you see, your learning, and what you would want to hear from teammates to change your mind.

Optional Activities

- A simple schema for the message envelope
- A short walkthrough of a single message's path

Constraints & Expectations

- Please spend **no more than 2–4 hours**
- Choose whatever format best communicates your thinking (doc, slides, diagram + text)
- Pseudo code is acceptable
- There is no single “correct” solution
- Assume clients may be on unreliable networks

What We’re Looking For

- How you break down an ambiguous problem
- How you reason about tradeoffs and risk
- How clearly you communicate your approach
- Whether your design feels practical for a real production system

If anything is unclear, make reasonable assumptions and call them out.

We are looking forward to seeing how you approach this challenge. Thank you for your time. Please email us if you have any questions.

Haiven Coding Challenge Disclaimer

By participating in this coding challenge, you agree to the following terms:

Purpose: This challenge is solely to evaluate your reasoning, thought process, system design, code quality, and communication skills as part of Haiven's hiring process. It is not compensated work. **Confidentiality:** All information provided in the challenge, including prompts, specifications, and related materials, is confidential and proprietary to Haiven. You must not disclose, share, or use it for any purpose other than completing the challenge. Upon request, return or destroy all such materials. **Intellectual Property Assignment:** Any code, designs, ideas, or other work product you create in response to this challenge is assigned to Haiven and becomes its exclusive property. You waive all rights, including moral rights, and agree not to use or claim ownership of such work product elsewhere.This disclaimer does not create an employment relationship. Haiven may use your submission internally for evaluation purposes only.

Agreed and accepted by: _____ Date: _____